PROJECT PROPOSAL

AWS IoT Monitoring and Control Dashboard

Cloud-based Smart Room Monitoring and Control

First Cloud Journey
July 2026

AWS IoT Monitoring and Control Dashboard

1. Executive Summary

The **AWS IoT Monitoring and Control Dashboard** addresses the need to monitor room conditions and control physical devices from a centralized interface. It connects YOLO UNO hardware to a backend deployed on AWS so that users can view temperature, humidity, and light readings, issue remote commands, and verify physical execution through command acknowledgement.

In the current Workshop deployment:

- YOLO UNO collects environmental telemetry and controls the fan, light, and curtain servo;
- CloudFront and AWS WAF deliver the React + Vite build from private Amazon S3;
- browser `/api/*` requests go through CloudFront to an Application Load Balancer;
- an Auto Scaling Group maintains two FastAPI EC2 instances running `aws-iot-backend.service`;
- Amazon RDS for PostgreSQL Multi-AZ stores telemetry and command states;
- YOLO UNO uses the ALB DNS name directly for telemetry, polling, and ACK;
- the device polls for commands and sends an ACK after execution; and
- Amazon CloudWatch collects backend logs, operational metrics, and alarm states.

The result is a reproducible Smart Room prototype that demonstrates an end-to-end flow from physical sensing to cloud storage, dashboard visualization, remote control, and physical-device acknowledgement.

2. Problem Statement and Target Users

2.1 Problem Statement

Small classrooms, laboratories, and equipment rooms often rely on sensors and actuators that operate independently. Values may only be visible at the device, historical records are not stored centrally, and operators cannot easily inspect room conditions or control equipment remotely. A dashboard action alone also does not prove that a fan, light, or curtain physically completed the requested operation.

Troubleshooting is fragmented when application logs, infrastructure metrics, database records, and device states are not connected. This project creates one traceable workflow for telemetry, commands, acknowledgement, and monitoring.

2.2 Target Users

Target user	Main need
Room or laboratory manager	Monitor environmental conditions and actuator status from one interface
Lecturers and students	Observe telemetry, historical data, and an end-to-end IoT experiment
Operators	Remotely control the fan, light, and curtain
Development team	Debug through logs, metrics, APIs, database records, and command states

2.3 Delivered Value

- A centralized dashboard for current values, historical telemetry, and controls.
- Two-way communication between the cloud backend and physical hardware.
- Persistent telemetry and command history in PostgreSQL.
- Physical-action verification through `Pending` and `Executed` states.
- Centralized logs and metrics for troubleshooting.
- A documented foundation that can later support additional rooms or devices.

---

3. Objectives and Scope

3.1 Hardware Objectives

- Read temperature and humidity from the DHT20 and read the analog light sensor.
- Control the fan, light/relay, and curtain servo; display status on LCD1602 I2C.
- Connect through Wi-Fi and HTTP REST.
- Support eight commands: `MODE_AUTO`, `MODE_MANUAL`, `FAN_ON`, `FAN_OFF`, `LIGHT_ON`, `LIGHT_OFF`, `CURTAIN_OPEN`, and `CURTAIN_CLOSE`.

3.2 Backend and Cloud Objectives

- Deploy FastAPI, Uvicorn, SQLAlchemy, and Pydantic on two ASG-managed Amazon EC2 instances behind an ALB.
- Manage the backend through `aws-iot-backend.service`.
- Store telemetry and commands in the `iot_dashboard` database on RDS PostgreSQL.
- Configure VPC networking, routing, and Security Groups for the deployment.
- Keep RDS non-public and allow PostgreSQL only from the EC2 Security Group.
- Use an EC2 IAM Role/Instance Profile for CloudWatch Agent permissions.
- Deliver the frontend from private S3 through CloudFront OAC and monitor it with AWS WAF managed rules.
- Use RDS PostgreSQL Multi-AZ with automated backups and a manual snapshot.

3.3 Frontend Objectives

- Display temperature, humidity, light, and historical charts through REST polling.
- Create fan, light, curtain, Manual, and Auto commands.
- Present threshold-based, rule-based analysis without describing it as machine learning.

3.4 Monitoring and Documentation Objectives

- Send backend logs to CloudWatch Logs.
- Monitor EC2 CPU, memory, disk, RDS CPU, and database connections.
- Configure alarms for important operational thresholds.
- Deliver bilingual Workshop instructions, source code, architecture, demo, and clean-up guidance.

3.5 Current Scope

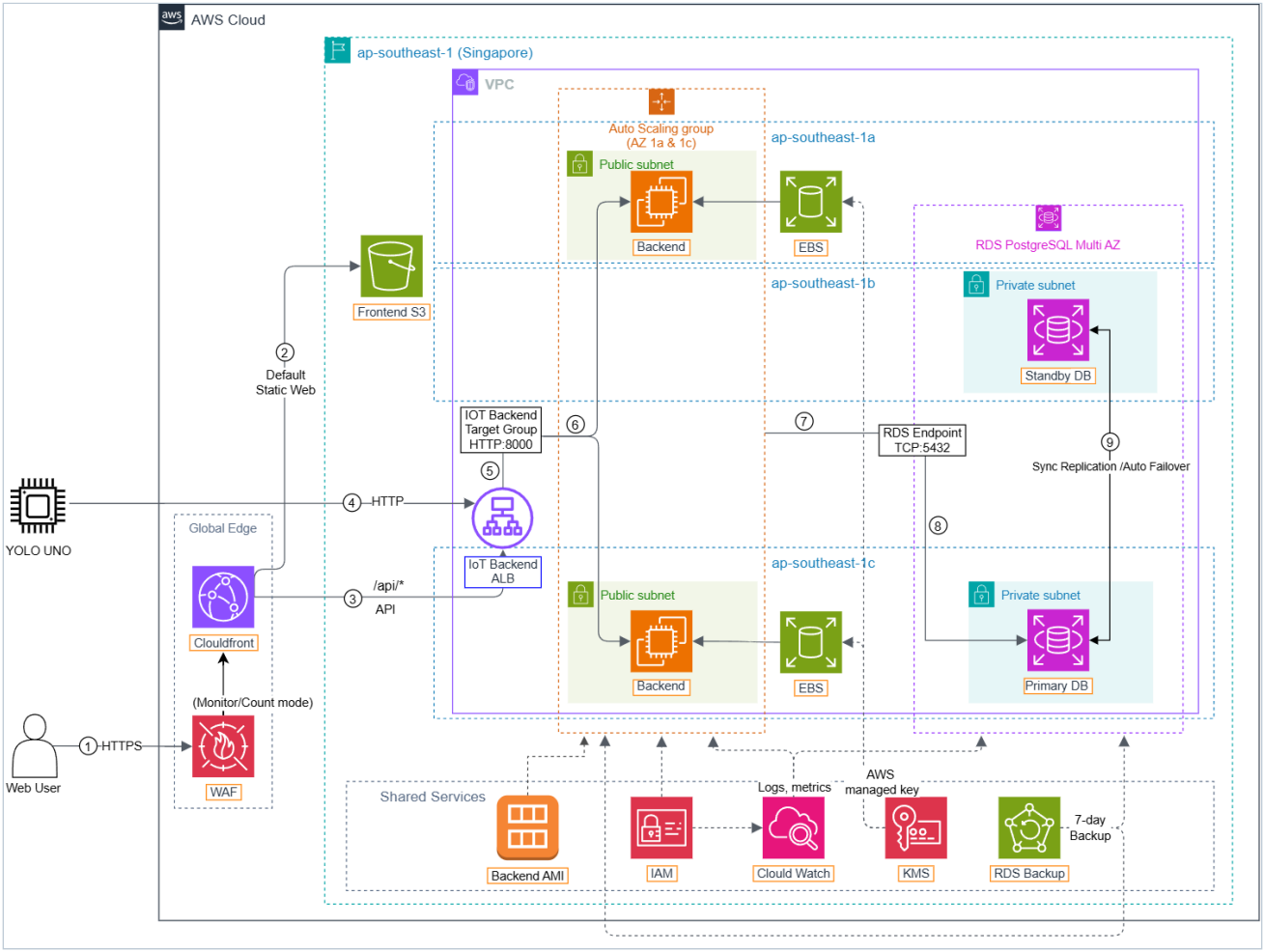
The prototype uses `room_01` as the logical identifier of the model room. It covers three environmental measurements, three physical actuators, HTTP REST, command polling, ACK, a CloudFront/WAF/private-S3 frontend, an ALB with an ASG of two FastAPI instances, RDS PostgreSQL Multi-AZ, and CloudWatch monitoring.

3.6 Out of Scope

The current delivery does not include a production multi-room rollout, user/device authentication, ALB origin HTTPS with a custom domain, load-based scaling policies beyond the configured ASG capacity, a tested failover drill, a mobile application, an automated delivery pipeline, or a trained machine-learning model. These capabilities require a separate review of requirements, cost, security, and operating complexity.

---

4. Solution Architecture



Current deployment architecture with CloudFront/WAF/private S3, ALB/ASG FastAPI backends, RDS PostgreSQL Multi-AZ, YOLO UNO, and CloudWatch.

4.1 Resource Placement

Location	Components	Responsibility
Outside AWS	Dashboard user, YOLO UNO ESP32-S3	User interaction; device telemetry, command execution, and ACK
AWS edge, outside VPC	CloudFront, AWS WAF, private S3 origin	Viewer HTTPS, static frontend delivery, `/api/*` routing, and WAF monitoring
AWS Cloud, outside VPC	EC2 IAM Role/Instance Profile, CloudWatch	Permissions, logs, metrics, dashboard, and alarms
Amazon VPC	Internet Gateway, two application subnets, private DB subnets	Network boundary and routing
Application subnets in 1a/1c	Internet-facing ALB, target group, ASG EC2 instances	HTTP entry point and FastAPI backend availability
Private DB subnets in 1c/1b	Amazon RDS for PostgreSQL Multi-AZ	Private telemetry/command persistence and standby failover

Location	Components	Responsibility
Attached to ASG EC2 instances	Encrypted 10 GiB gp3 EBS root volumes	OS, application, and runtime files

CloudWatch Agent runs on each backend instance and sends logs plus guest metrics to CloudWatch. ALB, ASG, EC2, and RDS publish managed metrics. The RDS standby is for failover and is not an application read replica.

4.2 Telemetry Flow

1. YOLO UNO reads the DHT20 and analog light sensor.
2. Firmware sends `POST /api/telemetry` directly to the ALB DNS name.
3. FastAPI validates and stores the record in RDS PostgreSQL.
4. The CloudFront-hosted frontend polls relative `/api/*` latest/history endpoints; CloudFront forwards them to the ALB.
5. The dashboard displays current readings and ordered history.

4.3 Command and Acknowledgement Flow

1. The operator creates a command from the CloudFront-hosted frontend through `/api/*`.
2. FastAPI stores it in the `Pending` state.
3. YOLO UNO polls the latest command for `room_01`.
4. Firmware executes the actuator or mode command.
5. YOLO UNO sends an ACK with the numeric command ID.
6. FastAPI updates the matching record to `Executed`.

4.4 Main REST Endpoints

Method	Endpoint	Purpose
`GET`	`/api/health`	Verify backend health
`POST`	`/api/telemetry`	Store telemetry
`GET`	`/api/devices/{device_id}/latest`	Retrieve latest telemetry
`GET`	`/api/devices/{device_id}/history`	Retrieve telemetry history
`POST`	`/api/devices/{device_id}/commands`	Create a command
`GET`	`/api/devices/{device_id}/commands/latest`	Poll the latest command
`POST`	`/api/devices/{device_id}/commands/{command_id}/ack`	Acknowledge execution

4.5 Security Boundary

- RDS has no public Internet access.
- The S3 frontend bucket has Block Public Access and is read through CloudFront OAC.

- CloudFront provides viewer HTTPS; the three WAF managed rule groups currently run in Count/Monitor mode.
- The ALB Security Group accepts HTTP:80; backend port 8000 accepts traffic from the ALB Security Group.
- RDS TCP `5432` accepts traffic from the EC2 Security Group, not a public CIDR.
- EC2 uses an IAM Role/Instance Profile for CloudWatch instead of static AWS credentials.
- Runtime values use environment or local secret files.
- `backend/.env`, `hardware/include/secrets.h`, `*.pem`, passwords, and credentials must not be committed.

---

5. Technical Implementation Plan

Phase 1 - Requirements and AWS Foundation

Confirm the Smart Room functional requirements and acceptance criteria, assign team responsibilities, and define `room_01` as the logical identifier. Then design the VPC/Security Group path, deploy the initial backend/database, and verify private database connectivity.

Phase 2 - Backend and Database

Define the PostgreSQL schema; implement health, telemetry, latest, history, command, polling, and ACK endpoints; configure environment values and systemd.

Phase 3 - Hardware and Firmware

Connect DHT20, the light sensor, fan, light/relay, servo, and LCD; develop PlatformIO firmware for sensing, Wi-Fi, telemetry, polling, actuator control, modes, and ACK; validate all eight commands.

Phase 4 - Frontend and Integration

Build the React + Vite + TypeScript dashboard, history charts, controls, and rule-based recommendations; deploy it to private S3 through CloudFront/WAF; add `/api/*` routing to the ALB; and resolve API, CORS, command-state, and hardware issues.

Phase 5 - Monitoring, Validation, and Handover

Create the backend AMI/Launch Template, ALB/target group/ASG, RDS Multi-AZ and backup controls; configure CloudWatch Agent, logs, metrics, and eight alarms; then complete end-to-end tests, security review, bilingual Workshop, evidence, demo, clean-up guide, and handover.

---

6. Timeline and Milestones

The Proposal follows the eight-week Worklog from **01/06/2026 to 31/07/2026**.

Week	Period	Main activities	Milestone
Week 1	01/06-07/06	Requirements, scope, initial architecture, roles, and plan	Agreed functions, acceptance scope, architecture, and assignments
Week 2	08/06-14/06	AWS architecture, VPC, IAM, and Security Groups	Reviewed cloud and security design
Week 3	15/06-21/06	EC2, EBS, RDS, network, and database connectivity	Running EC2 and available private PostgreSQL
Week 4	22/06-28/06	FastAPI, schema, environment, and systemd	Backend running and connected to RDS
Week 5	29/06-05/07	Telemetry, latest, history, command polling, and ACK APIs	Complete REST flow and persisted command states
Week 6	06/07-12/07	YOLO UNO firmware, sensors, actuators, Wi-Fi, and REST	Hardware sends telemetry and processes eight commands
Week 7	13/07-19/07	React dashboard, charts, controls, integration, and debugging	Frontend reads AWS data and creates commands
Week 8	20/07-31/07	CloudFront/WAF/S3, ALB/ASG, RDS Multi-AZ, end-to-end tests, CloudWatch, security, documentation, demo, and handover	Verified current architecture and submission-ready bilingual documentation

---

7. Resource Configuration, Cost and Optimization

7.1 Current Resource Configuration

Resource	Current configuration	Main cost factor	Optimization
S3 / CloudFront / WAF	Private S3 origin, CloudFront distribution, three WAF groups in Count mode	Storage, requests, transfer, plan/WAF usage	Keep only build artifacts; review plan limits and WAF mode
Application Load Balancer	Internet-facing, HTTP:80, two-AZ target group	Load balancer hours and capacity units	Remove after evaluation if no longer required
EC2 / Auto Scaling	Two `t3.micro` Linux instances, ASG `2/2/4`	Instance hours and data transfer	Review desired capacity and metrics; delete ASG after handover if approved

Resource	Current configuration	Main cost factor	Optimization
EBS	Encrypted 10 GiB `gp3` root volume per ASG instance	Provisioned storage and snapshots	Keep required storage and remove unused volumes/ snapshots
RDS PostgreSQL	`db.t4g.micro`, Multi-AZ, non-public	Multi-AZ instance hours, storage, backups, transfer	Review backup retention/ connections; delete after approved evidence retention
RDS storage	20 GiB General Purpose SSD	Provisioned storage	Monitor growth and avoid unnecessary retention
CloudWatch	Logs, ALB/ASG/EC2/RDS metrics, dashboard, eight alarms	Log ingestion/retention, custom metrics, dashboard, alarms	Set suitable retention and remove unused artifacts
Data transfer	CloudFront/API/device telemetry and polling	Internet, edge, and inter-AZ volume	Use reasonable intervals, compact payloads, and cache only static content
IAM/VPC foundation	Role, VPC, subnets, route, IGW, Security Groups	Basic configuration usually has no direct hourly cost; attached resources and processing may cost	Remove unused dependencies carefully

No fixed monthly total is claimed because charges depend on runtime, Region, traffic, retention, backups, and account pricing. Actual charges must be checked in AWS Billing and Cost Management.

7.2 Optimization Actions

- Use the dependency-aware clean-up procedure rather than stopping one ASG instance, which would be replaced while desired capacity remains 2.
- Delete RDS when the environment is retired; stopping is temporary and subject to service behavior.
- Use CloudWatch evidence to assess resource sizing.
- Retain logs only as long as needed for evaluation and troubleshooting.
- Balance polling responsiveness against request volume.
- Follow the dependency-aware clean-up order in Workshop section 5.10.

---

8. Risk Assessment

Risk	Likelihood	Impact	Mitigation
ALB origin/device route uses HTTP	High	High	Avoid sensitive payloads; add ACM/custom domain and origin/device TLS before broader use

Risk	Likelihood	Impact	Mitigation
WAF Count mode does not block	Medium	High	Review sampled requests and false positives, then stage tested Block mode
Security Group rules are too broad	Medium	High	Restrict SSH; allow backend 8000 only from ALB SG and RDS 5432 only from backend SG
Credentials are committed	Low	High	Use local secret files, placeholder examples, and review Git status
EC2 cannot reach RDS	Medium	High	Verify subnet, DB Subnet Group, DNS, SG rules, and TLS with `psql`
Device loses Wi-Fi/ backend connectivity	Medium	Medium	Add retry/reconnect logic and show connection state
Polling creates excess requests	Medium	Medium	Use documented intervals and review browser/ backend logs
Command remains `Pending`	Medium	High	Check polling, command name/ID, actuator result, ACK, and DB state
GPIO/power error causes instability	Medium	High	Use verified pin map, shared ground, safe power, and incremental tests
ASG deployment becomes inconsistent	Medium	High	Release with a versioned AMI/Launch Template and verify instance refresh/ target health
Multi-AZ failover is untested	Medium	High	Schedule a controlled failover drill and validate endpoint reconnection/ data integrity
Monitoring evidence is incomplete	Medium	Medium	Verify Agent, log streams, metric dimensions, retention, and eight alarms
Resources continue generating cost	Medium	High	Assign clean-up responsibility and follow dependency order

9. Expected Results and Success Criteria

9.1 Expected Results

The prototype should provide a traceable path from physical readings to AWS storage and dashboard visualization, plus a return path from dashboard commands to actuator execution and ACK. Evidence must allow another reviewer to inspect the application, database, hardware, and monitoring layers.

9.2 Measurable Success Criteria

Success criterion	Acceptance evidence
Backend runs	`systemctl status aws-iot-backend` shows `active (running)`
Health works	`/api/health` returns `status: ok`
CloudFront frontend works	Private-S3 React build loads through CloudFront viewer HTTPS
Browser API route works	CloudFront `/api/*` requests reach the ALB origin and return HTTP `200`
ALB/ASG is healthy	Two targets across two Availability Zones are Healthy and ASG desired capacity is 2
EC2 connects to RDS	`psql` uses SSL/TLS and `\dt` shows application tables
Telemetry persists	PostgreSQL contains a `room_01` telemetry record
Dashboard displays data	Temperature, humidity, and light show Live AWS
History works	Charts use `/api/devices/room_01/history`
Frontend calls succeed	Latest, history, and command requests return HTTP `200`
Commands persist	`commands` contains the created command and state
Actuators respond	Fan, light, and curtain servo react in demo evidence
ACK lifecycle works	The same command changes `Pending` → `Executed`
CloudWatch Logs works	Backend log stream contains FastAPI request logs
Metrics are visible	EC2 CPU/disk, RDS CPU/connections, and Agent-collected EC2 memory are shown
Alarms exist	CloudWatch lists eight project alarms
RDS Multi-AZ is enabled	CLI/console evidence identifies primary and standby Availability Zones
Backup controls exist	Automated retention is seven days and the manual snapshot is Available
RDS is private	Internet access is disabled or `Publicly accessible: No`

Success criterion	Acceptance evidence
PostgreSQL is restricted	RDS accepts TCP `5432` from the EC2 Security Group
Secrets stay out of Git	`.env`, `secrets.h`, `*.pem`, and real credentials are untracked

---

10. Current Limitations and Future Improvements

10.1 Current Limitations

- CloudFront provides viewer HTTPS, but the ALB origin and direct device path use HTTP and no custom domain is documented.
- API routes do not enforce strong user/device authentication or rate limiting.
- WAF managed rules are in Count/Monitor mode and have not been validated in Block mode.
- Polling-based command delivery and dashboard refresh.
- One model room identified by `room_01`.
- Rule-based recommendations, not trained machine learning.
- Deployment and testing remain mostly manual; no controlled ALB/ASG or RDS failover drill is documented.

10.2 Future Improvements

After reviewing requirements, security, cost, and complexity, future work may add custom-domain/ALB HTTPS, authentication, tested WAF blocking, multiple rooms and identities, failover drills, event-driven communication at greater scale, automated testing/deployment, managed secrets, operational notifications, and a mobile-friendly experience.

These are future options and are not part of the current deployment.

---

11. Team Responsibilities

Member	Role and responsibilities
Phạm Lê Minh Khôi	AWS infrastructure, EC2, EBS, RDS, VPC, Security Groups, IAM, CloudWatch, DevOps, PlatformIO firmware development, and YOLO UNO hardware integration
Thượng Đình Hưng	React + Vite frontend, dashboard, overall integration, debugging, and demo recording support
Ngô Minh Thuận	FastAPI backend, REST endpoints, PostgreSQL integration, telemetry, command processing, and ACK handling
Lê Bảo Khánh	Documentation and QA, Proposal, Blogs, Worklog, Event Reports, and bilingual Workshop content

12. Deliverables and Evidence

Deliverable	Content	Evidence
GitHub repository	Backend, frontend, hardware, diagrams, bilingual README	**Source Code**, public `main`
FastAPI backend	Health, telemetry, history, command, polling, ACK	**Backend Workshop**, systemd and health evidence
PostgreSQL database	Tables, telemetry, command states	`psql`, `\dt`, telemetry and command queries
React + Vite frontend	Telemetry, controls, rules, history charts	**Frontend Workshop**, screenshots
YOLO UNO firmware	Sensors, actuators, modes, polling, ACK	**Hardware Workshop**, source, build, demo
End-to-end validation	Telemetry persistence, lifecycle, physical execution	**Testing Workshop**, **Demo Video**
AWS edge and availability	CloudFront/WAF/private S3, ALB/ target group/ASG, RDS Multi-AZ and backups	**AWS Workshop**, screenshots
CloudWatch monitoring	Logs, ALB/ASG/EC2/RDS metrics, dashboard, eight alarms	**CloudWatch Workshop**, screenshots
Bilingual Workshop	Deployment, integration, testing, monitoring, handover	**Workshop**
Clean-up guide	Inventory, cost, security, dependency-aware removal	**Cost, Security, Clean-up**
Reference set	Source, demo, documents, technical links	**References**

Public project links:

- Project source code
- End-to-end demo video